

Autonomous Emergency Braking System (AEBS)

System Specification and Formal Abstraction

Heba Al Kayed

Abstract

This document formally specifies the dynamics of the AEBS case study. It is divided into three primary sections. **Part I** defines the concrete simulation environment, modelled as a Discrete-Time Stochastic Hybrid System (dt-SHS), detailing the physical plant, sensor noise, and controller logic. **Part II** details the formal abstraction pipeline, which discretizes the concrete dynamics into an Interval Markov Decision Process (IMDP) for formal verification using the PRISM model checker. **Part III** presents the formal verification results, empirically analyzing algorithmic limitations and boundary conditions under varying uncertainty profiles.

Part I: Concrete System Dynamics

1 System Architecture

The system is modelled as a closed-loop simulation consisting of four distinct components:

- **Vehicle Plant ($\mathcal{P}$):** A discrete-time physical model of the ego vehicle.
- **Traffic Environment ($\mathcal{E}$):** Manages the dynamics of the lead vehicle and scenario configuration.
- **Perception System ($\mathcal{S}$):** Acts as a stochastic observer function $\mathbf{y} = h(\mathbf{s})$.
- **Controller ($\mathcal{C}$):** A deterministic reactive policy mapping observations to control inputs.

2 Vehicle Plant Dynamics

The vehicle dynamics are implemented in the `VehiclePlant` class. The system state evolves in discrete time steps of $\Delta t = 0.1\text{s}$.

2.1 State Space Definition

The state vector is defined as $\mathbf{s}_k = [x_k, v_k, a_k]^\top$, where:

- x_k : Position (or Gap) at step k .
- v_k : Velocity (or Closing Speed) at step k .
- a_k : Longitudinal Acceleration at step k .

The system supports two coordinate references:

- **World Frame:** x denotes absolute position on the track (increasing).
- **Relative Frame:** x denotes the **Gap** to the obstacle (decreasing). This frame is utilized for formal abstraction.

2.2 Discrete-Time Update Law

The physics engine implements a **sequential discrete-time update** scheme. In this approach, each state variable is computed sequentially and immediately utilized in the subsequent equations within the same time step (t_{k+1}). This forward-chaining ensures that velocity incorporates the most recent acceleration, and position incorporates the most recent velocity, improving the physical consistency of the simulation.

The state update equations for the transition $t_k \rightarrow t_{k+1}$ are executed sequentially as follows:

1. Acceleration Update (Actuator Lag)

$$a_{k+1} = a_k + \alpha \cdot (u_k - a_k) \quad (1)$$

Where:

- a_{k+1}, a_k : The actual longitudinal acceleration of the ego vehicle at step $k + 1$ and k , respectively.
- u_k : The numeric control input requested by the controller at step k . This is a continuous value representing the target longitudinal acceleration in m/s^2 (e.g., $u_k = -9.8$ for emergency braking).
- α : The actuator lag coefficient ($\alpha = 0.5$), representing the physical delay in the braking system's response.

Explanation: Real-world vehicles cannot change acceleration instantaneously due to hydraulic and mechanical delays. Equation 1 acts as a discrete-time first-order low-pass filter to model this. If a vehicle is coasting ($a_k = 0.0 \text{ m/s}^2$) and the controller suddenly commands an emergency stop ($u_k = -9.8 \text{ m/s}^2$), the actual acceleration in the first 0.1s time step becomes:

$$a_{k+1} = 0.0 + 0.5 \cdot (-9.8 - 0.0) = -4.9 \text{ m/s}^2$$

Assuming the controller continues to hold this emergency command in subsequent steps ($u_{k+1} = -9.8$, $u_{k+2} = -9.8$), the actual acceleration asymptotically approaches the target. In the second step, it becomes $a_{k+2} = -4.9 + 0.5 \cdot (-9.8 - (-4.9)) = -7.35 \text{ m/s}^2$. This mathematically mirrors the physical accumulation of hydraulic brake pressure over time.

2. Kinematic Velocity Update

$$v_{k+1} = \max(0, v_k + a_{k+1} \cdot \Delta t) \quad (2)$$

Where:

- v_{k+1}, v_k : The relative velocity of the vehicle at step $k + 1$ and k .
- a_{k+1} : The newly calculated actual acceleration from Equation 1.
- Δt : The time step duration (0.1s).

Explanation: This computes the new closing speed based on standard kinematics ($v = v_0 + at$), but distinctly utilizes the *newly calculated* acceleration (a_{k+1}) from Equation 1. The outer $\max(0, \dots)$ function acts as a physical boundary constraint: when a vehicle brakes to a complete stop, it does not begin accelerating backwards. This mathematically guarantees the velocity cannot drop below zero.

3. Relative Position Update

$$x_{k+1} = \begin{cases} x_k + v_{k+1} \cdot \Delta t & \text{if world_frame} \\ x_k - v_{k+1} \cdot \Delta t & \text{if relative_frame} \end{cases} \quad (3)$$

Where:

- x_{k+1}, x_k : The position of the vehicle or gap to the lead vehicle at step $k + 1$ and k .
- v_{k+1} : The newly calculated actual velocity from Equation 2.
- Δt : The time step duration (0.1s).

Explanation: Finally, the position is updated using the *newly calculated* velocity (v_{k+1}), completing the forward-chaining sequence. For the formal verification abstraction, the model operates in the **relative_frame**. In this frame, x represents the spatial **gap** between the ego vehicle and the obstacle. Therefore, a positive closing speed (v_{k+1}) actively *reduces* the gap distance over the time step Δt , resulting in a subtraction operation.

```

1 # 1. Update Acceleration (First-Order Lag)
2 # Maps to Eq (1): a_new = a + alpha * (u - a)
3 self.actual_acceleration = self.actual_acceleration + self.alpha * (action_acceleration
4   - self.actual_acceleration)
5
6 # 2. Update Velocity
7 # Maps to Eq (2): v_new = v + a_new * dt
8 self.actual_velocity += self.actual_acceleration * self.dt
9
10 # Constraints: No negative velocity
11 if self.actual_velocity < 0.0:
12     self.actual_velocity = 0.0
13     self.actual_acceleration = 0.0
14
15 # 3. Update Position
16 # Maps to Eq (3): x_new = x + v_new * dt
17 if self.coordinate_system == 'world_frame':
18     self.x += self.actual_velocity * self.dt
19 else:
20     # Relative Frame: x is GAP. Positive closing speed REDUCES the gap.
21     self.x -= self.actual_velocity * self.dt

```

Listing 1: Implementation of Sequential Forward-Chaining Update

3 Perception and Sensor Noise Model

The `PerceptionSystem` implements the observation function $h : \mathcal{S} \rightarrow \mathcal{Y} \cup \{\emptyset\}$. It maps the ground truth state $\mathbf{s}_k$ to a noisy measurement $\mathbf{y}_k = [\tilde{x}, \tilde{v}]^\top$ or a null detection ($\emptyset$). The model captures two distinct types of uncertainty that mirror real-world autonomous driving sensor systems: detection uncertainty (classification) and measurement uncertainty (regression).

3.1 Observation Function

The logic operates across three distinct regimes to map the ground truth state to the observed state:

$$\mathbf{y}_k = \begin{cases} \emptyset & \text{if } x_k > d_{max} \quad (\text{Out of range}) \\ \emptyset & \text{with probability } P_{FN} \quad (\text{Missed detection}) \\ [\tilde{x}, \tilde{v}]^\top & \text{otherwise} \end{cases} \quad (4)$$

Where:

- $\mathbf{y}_k$: The observed state vector (the sensor reading) at time step k .
- $\emptyset$: A null detection state indicating the sensor suite failed to perceive a target.
- x_k : The ground truth relative gap to the target object.
- d_{max} : The maximum physical detection range limit of the sensor suite.
- P_{FN} : The probability of a false negative (missed detection) occurring.
- $\tilde{x}, \tilde{v}$: The noisy observed relative gap and relative closing velocity, respectively.
- **Deterministic Non-Detection:** If the ground truth gap x_k exceeds the hard sensor range limit ($d_{max} = 100.0\text{m}$), the system deterministically returns a "not detected" state. While premium front-facing automotive radars can detect objects up to 150–250m, 100.0m is utilized here as a deliberate conservative bound for the operational design domain.
- **Stochastic Detection Failure:** Within the detectable range ($x_k \leq d_{max}$), a stochastic miss probability (P_{FN}) is applied. Modeled as a Bernoulli process, this accurately simulates a Deep Neural Network (DNN) classifier failing to flag a target it should have caught.

3.2 Measurement Uncertainty and Truncation

For successful detections, the system introduces aleatoric measurement uncertainty via an additive zero-mean Gaussian noise vector $\mathbf{w}_k$:

$$\mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, \Sigma), \quad \Sigma = \begin{bmatrix} \sigma_{pos}^2 & 0 \\ 0 & \sigma_{vel}^2 \end{bmatrix} \quad (5)$$

Where:

- $\mathbf{w}_k$: The sampled multivariate noise vector at time step k .
- $\mathcal{N}$: The continuous Normal (Gaussian) probability distribution.
- $\mathbf{0}$: A zero-mean vector, implying the sensor is unbiased on average.
- Σ : The diagonal covariance matrix, assuming noise in gap and velocity are statistically independent.
- $\sigma_{pos}, \sigma_{vel}$: The standard deviations of the position and velocity measurement noise, respectively.

Crucially, to prevent physically impossible negative distances, the observed gap $\tilde{x}$ is subjected to non-negativity truncation, while the observed relative velocity $\tilde{v}$ remains unclamped (as an opening gap is physically valid):

$$\tilde{x} = \max(0, x_k + w_{pos}) \quad (6)$$

$$\tilde{v} = v_k + w_{vel} \quad (7)$$

Where:

- w_{pos} : The scalar position noise component drawn from the vector $\mathbf{w}_k$.
- w_{vel} : The scalar velocity noise component drawn from the vector $\mathbf{w}_k$.
- v_k : The ground truth relative closing speed of the ego vehicle in the relative coordinate frame (as defined in Section 2.1).

Because of the non-negativity truncation in Equation 6, the gap observation does not strictly follow a standard Normal distribution. Rather, the probability density function $f(\tilde{x})$ takes the form of a rectified Gaussian:

$$f(\tilde{x}) \propto \mathcal{N}(\tilde{x}; x_k, \sigma_{pos}^2) \cdot \mathbf{1}[\tilde{x} \geq 0] \quad (8)$$

Where:

- $f(\tilde{x})$: The probability density function of the observed gap.
- $\propto$: Proportional to, indicating that the resulting distribution requires renormalisation.
- $\mathcal{N}(\tilde{x}; x_k, \sigma_{pos}^2)$: The probability density of a Normal distribution evaluated at $\tilde{x}$, parameterized by mean x_k and variance σ_{pos}^2 .
- $\mathbf{1}[\tilde{x} \geq 0]$: The indicator function, which evaluates to 1 if $\tilde{x} \geq 0$ and 0 otherwise.

The proportional notation ($\propto$) highlights that renormalisation occurs inherently because the $\max(0, \dots)$ operation piles the probability mass of the negative tail identically onto zero as a point mass. This mathematically alters the mean and variance compared to the base noise distribution, especially when the true gap x_k approaches zero. This discrepancy is explicitly acknowledged, as it directly influences safety bounding during formal verification.

The baseline standard deviations ($\sigma_{pos} = 0.5\text{m}$ and $\sigma_{vel} = 0.1\text{m/s}$) represent the lower end of typical measurement tolerances for commercial 77 GHz Frequency-Modulated Continuous-Wave (FMCW) automotive radars, serving as an optimistic baseline **hasch'fmcw'2012**, **ti'radar'2020**.

3.3 Perception Modeling Limitations

This architecture establishes a clean, minimal uncertainty model well-suited for stress-testing the controller in a concrete simulation environment. However, it explicitly assumes the following constraints:

- **No False Positives:** The system can miss objects but never hallucinates non-existent obstacles (ghost targets).
- **Independent Noise:** Gap and velocity errors are modeled as independent distributions in the covariance matrix Σ . Realistically, a degraded radar return might positively correlate both errors simultaneously.
- **No Systematic Bias:** The Gaussian noise is strictly zero-mean. The model assumes the sensor is perfectly calibrated, ignoring real-world sensor drift or misalignment.
- **Stateless Execution:** Each sensor read evaluates independently. There is no Kalman-filter-style uncertainty propagation or temporal tracking logic over time.

```
1 # 1. Detection Logic (Classification DNN)
2 # Check range (Deterministic out-of-bounds)
3 if real_gap > 100.0:
4     return False, 100.0, 0.0
5
6 # Apply False Negative Rate (Bernoulli Miss Probability)
7 if np.random.rand() < self.fn_rate:
8     return False, real_gap, real_v_rel
9
10 # 2. Estimation Logic (Regression DNN)
11 # Add independent Gaussian noise to measurements
12 gap_noise = np.random.normal(0, self.pos_noise)
13 vel_noise = np.random.normal(0, self.vel_noise)
14
15 obs_gap = max(0.0, real_gap + gap_noise) # Truncated
16 obs_v_rel = real_v_rel + vel_noise      # Unclamped
```

Listing 2: Sensor Noise Injection

4 Controller Logic

The controller implements a reactive decision logic. It does not access the ground truth state $\mathbf{s}_k$; instead, it relies entirely on the noisy observation $\mathbf{y}_k$ and its own internal belief of its velocity state.

To support comprehensive benchmarking, the system implements two distinct parameter sets: an "Industry Mode" reflecting current automotive practice, and a more conservative "Safe Mode." The Industry Mode parameters are directly derived from the standard multi-stage collision mitigation algorithms widely utilized in current automotive research and safety assessment protocols. The time-to-collision (TTC) activation thresholds for Forward Collision Warning (2.6s), partial braking (1.6s), and full emergency braking (≤ 1.0 s), alongside their respective nominal deceleration limits ($\approx 0.4g$ and $1.0g$), align closely with the standard TTC baseline algorithms and Chinese national standards (GB/T 39901-2021) utilized in recent autonomous vehicle literature [1]. Furthermore, while standard TTC models often assume instantaneous step-changes in acceleration leading to passenger discomfort [1], our physical plant naturally mitigates abrupt jerk via the discrete-time first-order actuator lag (Equation 1).

Table 1: Controller Benchmark Parameters (Comparison)

Parameter	Symbol	Industry Mode	Safe Mode
<i>Time Thresholds</i>			
Warning TTC	TTC_{warn}	2.6s	6.0s
Braking TTC	TTC_{brake}	1.6s	5.0s
Emergency TTC	TTC_{crit}	1.0s	4.0s
<i>Distance Thresholds</i>			
Warning Dist	d_{warn}	10.0m	15.0m
Braking Dist	d_{brake}	6.0m	10.0m
Emergency Dist	d_{crit}	2.0m	5.0m
<i>Actuation</i>			
Brake Force	u_{brake}	-4.0 m/s ²	-4.0 m/s ²
Emergency Force	u_{crit}	-9.8 m/s ²	-8.0 m/s ²

Note: The Warning thresholds (TTC_{warn}, d_{warn}) are utilized strictly for system state labelling and formal verification safety monitoring, rather than triggering physical braking actuation.

The decision logic mathematically branches based on observed closing velocity ($v_{rel} < 5.0$ m/s) to distinguish between a low-speed approach (using spatial distance thresholds) and high-speed collision avoidance (using temporal TTC thresholds).

```

1 # 1. Update Internal Belief (Dead Reckoning)
2 # Note: est_velocity never receives external true-state updates
3 self.est_velocity = max(0.0, self.est_velocity + self.prev_accel_req * self.dt)
4
5 # 2. Compute Time-To-Collision (TTC)
6 # Guard against division by zero (opening gap or perfectly matching speed)
7 if obs_v_rel <= 0.0:
8     ttc = float('inf')
9 else:
10    ttc = obs_gap / obs_v_rel
11
12 # 3. Decision Logic (Threshold Branching)
13 if obs_v_rel < 5.0:
14     # Low-speed approach utilizes distance boundaries
15     if obs_gap < self.dist_emergency: return self.acc_emergency, 'emergency_brake'
16     elif obs_gap < self.dist_brake:   return self.acc_brake, 'brake'
17 else:
18     # High-speed approach utilizes TTC boundaries
19     if ttc < self.ttc_emergency:      return self.acc_emergency, 'emergency_brake'
20     elif ttc < self.ttc_brake:        return self.acc_brake, 'brake'
21
22 return 0.0, 'coast'

```

Listing 3: Reactive Decision Tree and Dead Reckoning

Implementation Note (Belief Drift): It is important to note that `est_velocity` relies purely on dead-reckoning (Euler integration of the previously requested acceleration) and is never synchronized with the external environment. Consequently, over extended horizons, this internal belief can silently drift from the true plant velocity. By default, internal belief noise is disabled to isolate environmental stochasticity during verification.

Part II: Abstracted System Dynamics

5 Formal Abstraction Methodology

Before detailing the computational optimization heuristics, it is necessary to establish the theoretical methodology of the formal abstraction pipeline. The fundamental challenge of verifying a discrete-time stochastic hybrid system (dt-SHS) lies in mapping a continuous, infinite state space into a finite representation that preserves formal safety guarantees.

Our methodology adapts the constructive procedure for finite approximate abstractions presented by Abate et al. [2]. By partitioning the continuous state space into a finite grid of cells with a bounded diameter δ , and integrating the stochastic transition kernel over these cells, the continuous system is abstracted into a finite-state Markov model.

Crucially, because the continuous kernel is being mapped to a discrete grid, the nominal integrated probabilities cannot perfectly capture the true continuous dynamics. The theory accounts for this by computing an explicit abstraction error bound, ϵ , derived from the grid resolution and the Lipschitz continuity constant of the stochastic kernel. Rather than producing a single transition matrix, the theoretical procedure outputs a **Markov Set-Chain (MSC)**. In an MSC, transitions between states are not scalar probabilities, but rather bounded sets or intervals $[p - \epsilon, p + \epsilon]$ guaranteed to contain the true continuous transition probabilities.

Because our target model checker, PRISM, does not natively support Markov Set-Chains, our methodology **re-encodes** the theoretical MSC output into a PRISM-compatible **Interval Markov Decision Process (IMDP)**. By mapping the probability intervals $[p - \epsilon, p + \epsilon]$ to the nondeterministic transition bounds of an IMDP, the model checker’s adversary naturally resolves the uncertainty. A pessimistic adversary maximizes the probability of collision by selecting the worst-case transition distribution within the bounds, ensuring that the verification yields a strict over-approximation of systemic risk.

Algorithm 1 defines the formal mathematical framework of this abstraction, while Algorithms 2 and 3 detail its computational execution.

5.1 Correspondence to Abate et al.

The computational implementation directly maps the constructive procedure of Abate et al. [2] into a software pipeline, as summarized in Algorithm 1. The continuous state space is geometrically partitioned (Step 1), the deterministic dynamics form a nominal successor around which the continuous stochastic kernel is integrated to find nominal discrete probabilities (Steps 3-4), and the explicit abstraction error ϵ is formulated (Step 5). The pipeline then re-encodes the resulting Markov Set-Chain intervals into IMDP syntax (Step 6) to allow for formal model checking. Where the software implementation structurally diverges from the pure mathematical theory—such as through Top- k pruning, asymmetric grid coarsening, and the isotropic noise assumption—these engineering generalizations and their impacts on formal soundness are explicitly detailed in Section 8.

Algorithm 1: Finite Modular IMDP Abstraction of a dt-SHS

Require: Concrete dt-SHS with state space $X \subseteq \mathbb{R}^3$, action space U , stochastic transition kernel T_x , and Lipschitz constant L

1. Select a finite partition of X into n rectangular cells: $X = \bigcup_{i=1}^n X_i$, each with diameter bounded by δ .
2. For each cell X_i , select the representative point $\bar{x}_i \in X_i$ as its geometric centre. Define the finite state set $\hat{X} := \{\bar{x}_i, i = 1, \dots, n\}$ augmented with absorbing boundary states $\{x_{crash}, x_{safe_sink}\}$.
3. For each representative action $\bar{u} \in \hat{U}$, apply the deterministic dynamics to obtain the nominal successor $\bar{x}' = f(\bar{x}_i, \bar{u})$ and its associated noise σ .
4. Compute nominal transition probabilities by integrating the stochastic kernel T_x over each target cell:

$$\hat{T}_x(X_j | \bar{x}_i, \bar{u}) = \int_{X_j} \mathcal{N}(x'; \bar{x}', \sigma^2) dx'$$

5. Compute the abstraction error bound: $\epsilon = V_{cell} \cdot L \cdot \|\text{radius}\|_2$
6. Construct IMDP transition intervals for all $\bar{x}_i, \bar{x}_j \in \hat{X}$:

$$\hat{T}_{IMDP}(X_j | \bar{x}_i, \bar{u}) = [\hat{T}_x - \epsilon, \hat{T}_x + \epsilon]$$

7. Assign safety labels to each cell: $\bar{x}_i \mapsto \{\text{crash}, \text{warning}, \text{safe}\}$ based on kinematic thresholds.
8. Compose the labelled plant IMDP with the controller and perception modules via synchronisation labels $\{\text{time_step}, \text{perceive}, \text{control}\}$.

Ensure: Composed modular IMDP $\hat{\Sigma} = (\hat{X}, \hat{U}, \hat{T}_{IMDP}, \hat{Y}, \hat{h})$ compatible with PRISM.

Algorithm 2: Transition Computation and Top- k Pruning

Require: Cell index i , grid $\mathcal{G}$, action space $\hat{U}$, dynamics wrapper f

1. Set $src \leftarrow \hat{X}_i$. If $src = x_{crash}$, return self-loop and halt.
2. Compute representative centre $\bar{x}_i \leftarrow \mathcal{G}.\text{centre}(i)$
3. **For each** action $\bar{u} \in \hat{U}$:
 - (a) Query dynamics: $(\bar{x}', \sigma, L) \leftarrow f(\bar{x}_i, \bar{u})$
 - (b) Compute error bound: $\epsilon \leftarrow V_{cell} \cdot L \cdot \|\text{radius}\|_2$
 - (c) Compute candidate transitions: $\mathcal{C} \leftarrow \text{ComputeKernel}(\bar{x}', \sigma, \mathcal{G})$ (Algorithm 3)
 - (d) Sort $\mathcal{C}$ by probability mass descending; retain top- k : $\mathcal{C}_k \leftarrow \mathcal{C}_{1:k}$
 - (e) Compute crash boundary mass: $p_{crash} \leftarrow \Phi\left(\frac{0 - \bar{x}'_{gap}}{\sigma}\right)$
 - (f) Compute residual mass: $p_{sink} \leftarrow \max(0, 1 - \sum_{c \in \mathcal{C}_k} c.p - p_{crash})$
 - (g) **For each** $c \in \mathcal{C}_k$, add IMDP interval:
 $\hat{T}_{IMDP}(X_c | \bar{x}_i, \bar{u}) = [\max(0, c.p - \epsilon), \min(1, c.p + \epsilon)]$
 - (h) Route crash and sink states with intervals $[p - \epsilon, p + \epsilon]$

Ensure: IMDP transition intervals for all actions from state $\bar{x}_i$

Algorithm 3: Gaussian Kernel Integration

Require: Nominal successor $\bar{x}'$, isotropic noise scalar σ , grid $\mathcal{G}$

1. Define search radius $r \leftarrow 4\sigma$ to capture $> 99.99\%$ of Gaussian mass
2. Enumerate all cells $\mathcal{R} \leftarrow \mathcal{G}.\text{intersect}(\bar{x}', r)$ within the search radius
3. Initialise candidate set $\mathcal{C} \leftarrow \emptyset$
4. **For each** cell $X_j \in \mathcal{R}$:
 - (a) Initialise $p \leftarrow 1$
 - (b) **For each** dimension $d \in \{gap, vel, accel\}$:

$$p \leftarrow p \cdot \left[\Phi \left(\frac{high_d - \bar{x}'_d}{\sigma} \right) - \Phi \left(\frac{low_d - \bar{x}'_d}{\sigma} \right) \right]$$

- (c) If $p > 0$, append (X_j, p) to $\mathcal{C}$
5. Return $\mathcal{C}$

Ensure: Set of candidate transitions $\mathcal{C}$ with associated probability masses

6 Computational Challenges and Optimizations

While the theoretical abstraction provides a sound mathematical framework, executing the generated IMDP in the PRISM model checker introduced severe computational and memory bottlenecks. The primary verification environment was constrained to a local machine with a 12GB RAM limit. To successfully verify the continuous physical model within these constraints, we introduced critical algorithmic optimizations.

The lead vehicle's stochastic acceleration uncertainty is incorporated directly into the plant's transition kernel. The perception module is retained as a symbolic identity stub ($y'=s$) to preserve the modular PRISM architecture while avoiding the state-space cost of explicit observation tracking.

6.1 PRISM Synchronization Protocol

To faithfully reproduce the sequential nature of the closed-loop simulation within the PRISM model, a specific synchronization protocol is enforced. This architectural constraint enforces the causal ordering of the closed-loop cycle, ensuring the controller always acts on the most recently observed state.

The modules synchronize via a shared global variable $t \in \{1, 2, 3\}$, which cycles through the system components in a strict temporal order:

```
1 def _write_turn_module(self, f):
2     f.write("module Turn\n")
3     f.write("    t : [1..3] init 1;\n")
4     f.write("    [time_step] (t=1) -> (t'=2);\n") # Plant acts
5     f.write("    [perceive] (t=2) -> (t'=3);\n") # Sensors acts
6     f.write("    [control] (t=3) -> (t'=1);\n") # Controller acts
7     f.write("endmodule\n")
```

Listing 4: PRISM Module Synchronization (from `prism_generator.py`)

6.2 Asymmetric Grid Coarsening

The state space is defined by the domains of relative distance, closing speed, and acceleration. To capture the extreme physics of an imminent collision without "tunneling" (probability mass leaking out of the physical simulation bounds), the acceleration limits required widening to $[-15, 10] \text{ m/s}^2$. Initially, this ballooned the system to over 623,000 states, resulting in JVM `OutOfMemoryError` failures during explicit model checking.

To resolve this, we employed asymmetric coarsening: the spatial and velocity resolutions were kept fine, but the acceleration resolution was doubled from 0.25 to 0.5. This safely reduced the state space to approximately 300,000 states without sacrificing critical spatial precision, satisfying the 12GB memory limit.

6.3 Top- k Transition Pruning

Mapping continuous Gaussian distributions to a discrete grid creates an overwhelmingly dense transition matrix. Without pruning, even within the bounded 4σ search radius, the Cartesian product of cells across three dimensions produces a candidate set large enough to cause transition explosion in PRISM. To prevent a transition explosion, we implemented a Top- k pruning strategy within the integration step.

The algorithm sorts the calculated target candidates by probability mass and retains only the top $k = 3$ most likely target states per transition. The discarded probability mass is routed directly to the `safe_sink` state, which is then subjected to the generalized uncertainty bounds (ϵ). While this drastically reduces the model checker’s branching factor, it relies on treating the discarded tail mass as structurally safe.

Needed Improvement: Strict Soundness in Pruning. Currently, the abstraction error ϵ is computed theoretically prior to pruning. A required future enhancement is that the ϵ bound on the absorbing sink states should be augmented by the exact discarded mass to ensure formal soundness, rather than relying solely on a pre-pruning Lipschitz bound that may underestimate the true residual uncertainty.

7 Theoretical Deviations and Practical Generalizations

While the abstraction pipeline fundamentally mirrors the constructive procedure outlined by Abate et al. [2], several theoretical deviations and engineering generalizations were required to transition from pure mathematics to a computationally tractable software implementation. These adaptations fall into two distinct categories: deliberate engineering generalizations that maintain theoretical soundness (e.g., the error bound generalization and ergodicity considerations), and heuristic deviations that compromise strict formal soundness to achieve computational tractability (e.g., Top- k pruning and the isotropic noise assumption).

- **Top- k Pruning Soundness:** The theoretical definition of a Markov Set-Chain (MSC) assumes full integration of the probability kernel. The implemented Top- k pruning (retaining only the 3 most likely transitions) drops the infinite tails of the Gaussian distribution. As acknowledged in Section 7.3, this slightly compromises the rigorous bounds of the MSC abstraction (and the resulting IMDP) if the discarded mass exceeds ϵ .
- **Ergodicity and Absorbing States:** The asymptotic convergence guarantees provided in the reference literature require the discrete-time stochastic hybrid system to be strongly aperiodic and irreducible (i.e., every state is reachable from every other state, and the system does not get trapped in cyclic, periodic loops). Because our formal model is designed for safety verification rather than steady-state analysis, it explicitly contains strictly absorbing states (e.g., `crash` and `safe_sink`). Consequently, the system is intentionally non-ergodic. The verification relies entirely on the single-step error bounds rather than infinite-horizon convergence theorems.

Part III: Formal Verification Results

8 Sensitivity Analysis and Bounds Formulation

The generated IMDP was verified using the PRISM model checker’s explicit engine. Because the abstraction utilizes intervals, queries are resolved by an *Adversary* evaluating reachability properties.

The four kinematic scenarios in Table 2 correspond to specific discrete states in the abstracted grid. Because the PRISM model declares the initial state as an undetermined integer constant `start_s` rather than a fixed value, any grid state can be used as a starting point without recompiling the model. The abstraction pipeline resolves each continuous scenario tuple (gap, velocity, acceleration) to its nearest grid cell and prints the corresponding flat state index at generation time:

```

1 =====
2 SCENARIO ID CHEAT SHEET (Use these in PRISM -const)
3 =====
4 SCENARIO | PHYSICS (Gap, V, A) | ID
5 -----
6 Nominal_Cruising | (40.0, 15.0, 0.0) | <id>
7 Warning_Threshold | (25.0, 15.0, 0.0) | <id>
8 Critical_Braking | (15.0, 15.0, 0.0) | <id>
9 Imminent_Collision | (10.0, 20.0, 0.0) | <id>
10 Post_Collision | (0.0, 0.0, 0.0) | 0
11 =====

```

Listing 5: Console Output: Scenario ID Cheat Sheet

Each scenario is then verified by supplying the printed ID to PRISM at the command line:

```

1 prism modular_system.prism -const start_s=<ID> -pf 'Pmax=? [ F "crash" ]'

```

Listing 6: Example PRISM Execution Command

This approach allows the single compiled model to be queried from any initial condition without regeneration, and ensures the continuous kinematic values in Table 2 map unambiguously to discrete grid states. Furthermore, the inclusion of the `Post.Collision` scenario implicitly confirms the structural integrity test detailed in Section 9.4, as the physical coordinates (0.0, 0.0, 0.0) map directly to state 0—the strictly absorbing crash state.

- P_{max} (**Worst-Case**): A pessimistic adversary maximizes the probability of collision, revealing the absolute upper bound of systemic risk.
- P_{min} (**Best-Case**): An optimistic adversary minimizes the probability, revealing the baseline risk imposed purely by physical momentum.

To empirically validate the robustness of the formal abstraction, a sensitivity analysis was conducted by varying the standard deviation (σ) of the lead vehicle’s acceleration noise (implemented as `lead_noise_std` in m/s^2). Crucially, because σ is baked directly into the transition kernel during the formal integration step (Algorithm 3), it is not a PRISM constant. Altering the noise profile to test $\sigma = 0.1$ or $\sigma = 4.0$ requires fundamentally recompiling the abstraction pipeline to generate a new IMDP matrix. Within the continuous dynamics, this acceleration variance approximately scales into the velocity domain as $\sigma_{vel} \approx \sigma \cdot \Delta t$, assuming single-step propagation through the actuator lag filter.

The baseline for these experiments was established at $\sigma = 2.0 \text{ m/s}^2$, diverging from the unconstrained upper-bound default of 4.0 m/s^2 . This established a physically recoverable middle ground—a variance regime where the vehicle’s physical braking capacity provides sufficient time and distance to avoid a crash under normal conditions, preventing the system from artificially failing out of the gate while still introducing meaningful noise. This testing revealed critical insights into how finite approximate abstractions handle boundary conditions versus strictly unrecoverable states under varying degrees of uncertainty.

Table 2: Sensitivity Analysis: Bounded Crash Probabilities $[P_{min}, P_{max}]$ across Noise Profiles ($\sigma = \text{lead_noise_std}$)

Scenario	Kinematics (Gap, V)	Low Noise ($\sigma = 0.1$)	Baseline ($\sigma = 2.0$)	High Noise ($\sigma = 4.0$)
Nominal Cruising	40m, 15m/s	[0.0, 0.0]	[0.0, 0.0]	[0.0, 0.0]
Warning Threshold	25m, 15m/s	$[1.27 \times 10^{-29}, 0.0002\%]$	$[6.55 \times 10^{-29}, 0.0005\%]$	$[4.23 \times 10^{-114}, \approx 0.0]$
Critical Braking	15m, 15m/s	[0.078%, 90.22%]	[0.0002%, 27.16%]	$[1.80 \times 10^{-29}, 0.021\%]$
Imminent Collision	10m, 20m/s	[35.94%, 99.999%]	[8.21%, 78.70%]	[0.013%, 4.13%]

8.1 Tightening Bounds under Low Uncertainty

When the physical noise is heavily constrained ($\sigma = 0.1$), the continuous Gaussian stochastic kernel acts as a narrow spike. In the *Critical Braking* scenario (15m gap, 15m/s), reducing the noise caused the worst-case crash probability (P_{max}) to skyrocket from the baseline 27.16% to **90.22%**. Because the variance was tiny, the narrow kernel concentrated mass into very few cells, meaning the Top- k pruning algorithm discarded negligible probability. Independently, the reduced σ yields a smaller Lipschitz constant L ,

which tightens ϵ and narrows the IMDP intervals. However, this effect is highly geometry-dependent. For instance, in the *Warning Threshold* scenario, the tight kernel actually reduces P_{max} from 0.0005% to 0.0002%, because the controller has sufficient spatial margin to safely resolve the scenario regardless of noise. The sky-rocketing risk in the *Critical Braking* scenario reveals a harsh physical reality: without variance-induced probability smearing into adjacent safer cells, the true kinematic outcome of a marginal physical state dominates, making a crash highly likely.

8.2 False Hope in Unrecoverable States

Analyzing the strictly unrecoverable *Imminent Collision* scenario (10m gap, 20m/s) under baseline noise ($\sigma = 2.0$) exposes a phenomenon we term the "False Hope" window. Although the vehicle lacks the physical actuator authority to avoid a collision, the wide variance of the stochastic kernel smears probability mass into adjacent grid cells representing safer states. The model checker interprets this mathematical smearing as a **21.30%** chance of survival ($P_{max} = 78.70\%$), as the optimistic adversary selects the transition distribution within each interval that minimizes crash probability, mathematically allowing probability mass to route through safer cells. Decreasing σ eliminates this smearing, correctly driving the crash probability toward 100%.

8.3 Probability Mass Dissipation under High Uncertainty

Conversely, artificially inflating the noise ($\sigma = 4.0$) triggered a mathematical paradox: the P_{max} for the *Critical Braking* scenario erroneously plummeted to **0.021%**.

This anomaly does not reflect increased physical safety, but rather exposes the boundary conditions of the abstraction algorithm under strict computational memory constraints. Inflating the variance severely flattens the stochastic kernel. Consequently, the Top-3 pruning strategy ($k = 3$) discards the vast majority of the probability mass residing in the wide tails. Furthermore, the flattened distribution yields a drastically reduced Lipschitz constant (L), resulting in an ϵ bound that is too small to mathematically compensate for the heavily pruned mass.

The result is probability mass dissipation from the abstracted transition matrix. Notably, under high noise, the interval width ($P_{max} - P_{min}$) for *Critical Braking* collapses completely, from $\sim 27\%$ in the baseline down to a mere $\sim 0.021\%$, anchoring tightly near zero. This interval collapse is a stronger diagnostic of abstraction breakdown than simply a suppressed upper bound. This finding rigorously validates the necessity of the algorithmic fix proposed in Section 7.3: computing ϵ post-pruning and explicitly routing the discarded mass to the sink state. While executing the abstraction on a high-memory compute cluster with a dynamically scaled retention threshold (k) offers a secondary mitigation, patching the probability leak at the algorithmic level is paramount.

8.4 Structural Integrity of the Abstraction

To guarantee that the abstraction does not conflate crash events with safe boundary exits, we verified the structural isolation of the crash state. Model checking the property `Pmax=? [ F "safe_sink" ]` from the initial crash state (state index 0, as defined in Algorithm 2) yielded a probability of exactly 0.0. This proves that the crash state is strictly absorbing, and probability mass cannot leak into the safe sink post-collision.

A Python Implementation Listings

```

1 FUNCTION ProcessStateTransitions(idx, G, action_space, wrapper):
2     src_id <- G.FlatIndex(idx)
3     IF src_id == 0: RETURN SelfLoop(0) // Crash is strictly absorbing
4
5     center <- G.IndexToCellCenter(idx)
6
7     FOR EACH (act_id, act_val) IN action_space:
8         // Step 1: Query Physics (Deterministic transition and noise)
9         next_center, sigma, L <- wrapper.GetNextStateDistribution(center, act_val)
10
11         // Step 2: Compute Formal Error Bound (Lipschitz error bound)
12         epsilon <- G.cell_volume * L * G.cell_radius
13

```

```

14 // Step 3: Integrate Stochastic Kernel
15 candidates <- ComputeKernel(next_center, sigma, G)
16
17 // Step 4: Top-k Pruning (Engineering Approximation)
18 Sort candidates by probability (descending)
19 top_k <- candidates[0:3]
20
21 // Step 5: Boundary Probabilities
22 p_crash <- GaussianCDF(gap_wall, next_center[0], sigma)
23 total_captured <- Sum(c.prob for c IN top_k)
24 p_safe_sink <- max(0, 1 - total_captured - p_crash)
25
26 // Step 6: Construct IMPD Intervals
27 FOR EACH c IN top_k:
28   p_min <- max(1e-6, c.prob - epsilon)
29   p_max <- min(1.0, c.prob + epsilon)
30   imdp.AddTransition(src_id, act_id, c.id, [p_min, p_max])
31
32 Route to Crash and Safe Sink with [p - epsilon, p + epsilon]

```

Listing 7: Computational Implementation of Algorithm 2 (Transition Computation and Top-k Pruning)

```

1 FUNCTION ComputeKernel(next_center, sigma, G):
2   // Define bounds to capture >99.99% of Gaussian mass
3   radius <- 4 * sigma
4   ranges <- G.GetIntersectingCells(next_center, radius)
5   candidates <- []
6
7   FOR EACH target_idx IN CartesianProduct(ranges):
8     prob <- 1.0
9     FOR EACH dimension d IN [0, 1, 2]:
10      low, high <- G.CellBounds(target_idx, d)
11      IF sigma > 0:
12        prob *= GaussianCDF(high, next_center[d], sigma)
13              - GaussianCDF(low, next_center[d], sigma)
14      ELSE:
15        prob *= (1.0 IF low <= next_center[d] <= high ELSE 0.0)
16
17   IF prob > 0:
18     candidates.Append({id: G.FlatIndex(target_idx), prob: prob})
19
20 RETURN candidates

```

Listing 8: Computational Implementation of Algorithm 3 (Gaussian Kernel Integration)

References

- [1] F. Lai, J. Liu and Y. Hu, “An automatic emergency braking control method for improving ride comfort”, *World Electric Vehicle Journal*, vol. 15, no. 6, p. 259, 2024. DOI: 10.3390/wevj15060259.
- [2] A. Abate, A. D’Innocenzo and M. D. Di Benedetto, “Approximate abstractions of stochastic hybrid systems”, *IEEE Transactions on Automatic Control*, vol. 56, no. 11, pp. 2688–2694, 2011. DOI: 10.1109/TAC.2011.2160595.